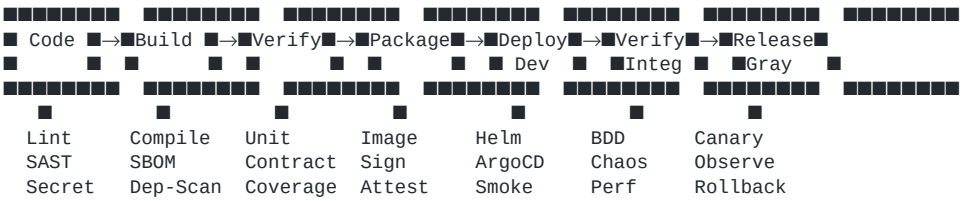
09 · DevOps 交付流水线与工程规范

项	内容
文档编号	DT-PdM-09
依据	Harness.io（CI/CD 平台）；GitOps（ArgoCD）；SLSA 供应链安全；GB/T 8567 配置管理
版本	v0.1

> 再次澄清：本文的 Harness 指 Harness.io 软件交付平台；智能体运行时工程见 `docs/05`。

1. 交付流水线总览



核心原则：

- 一切皆代码（流水线、环境、配置、判据、提示词、技能）
- 制品不可变（镜像 digest 引用，不用 tag）
- 环境晋升而非重建（同一制品从 dev → staging → prod）
- 每次变更可回滚（< 15 分钟）

2. Harness.io 流水线设计

2.1 流水线清单

流水线	触发	阶段	时长目标
`pr-verify`	PR 创建/更新	Lint → SAST → Unit → Contract	≤ 8 min
`main-build`	合并主干	Build → SBOM → 镜像 → 签名 → 推送	≤ 12 min
`deploy-dev`	构建成功	Helm 部署 dev → 冒烟	≤ 5 min
`verify-integration`	dev 部署成功	BDD → 混沌 C1 – C3 → 性能基线	≤ 30 min
`deploy-staging`	集成通过	部署 staging → 全量回归	≤ 40 min
`release-canary`	人工批准	灰度 5% → 观测 24h → 25% → 100%	按策略
`rollback`	手动/自动	回滚到上一 digest	≤ 5 min

2.2 关键阶段配置（示例）

```
# .harness/pipelines/pr-verify.yaml
pipeline:
  name: pr-verify
```

```
stages:
  - stage: static
    steps:
      - run: ruff check . && mypy src/
      - run: bandit -r src/ -ll
      - run: gitleaks detect --no-git
  - stage: test
    steps:
      - run: pytest tests/unit --cov=src --cov-fail-under=80
      - run: pytest tests/contract
  - stage: security
    steps:
      - run: trivy fs --severity HIGH,CRITICAL --exit-code 1 .
```

2.3 质量门禁（阻断式）

门禁	阈值	阻断点
单测覆盖率	< 80%	PR 合并
覆盖率下降	任意下降	PR 合并
SAST 高危	> 0	PR 合并
密钥泄露	> 0	PR 合并（并告警）
依赖漏洞 CVSS ≥ 7	> 0 未修复	构建
镜像高危	> 0	部署
BDD 失败	> 0	进 staging
性能回退	P95 劣化 > 20%	进 staging
模型评测回退	召回下降 > 3% 或误报上升 > 50%	上线

> "模型评测回退"是 AI 项目特有的门禁。没有它，模型迭代就是开盲盒。

---

3. 仓库结构（Monorepo + 独立部署）

```
diting-pdm/
■■■ README.md
■■■ docs/                          # ■■■■
■   ■■■ adr/                       # ■■■■■■■
■   ■■■ ...
■■■ specs/                         # SDD ■■■■■■■■■■
■   ■■■ constitution.md
■   ■■■ 001-■■■■■■■/
■   ■   ■■■ spec.md
■   ■   ■■■ plan.md
■   ■   ■■■ tasks.md
■   ■   ■■■ features/*.feature
■   ■■■ 002-■■■■■■■/
■■■ services/
■   ■■■ ingest-gateway/            # Go
■   ■■■ feature-engine/            # Python
■   ■■■ diagnosis-engine/          # Python
■   ■■■ prognosis-engine/          # Python
■   ■■■ agent-runtime/             # Python■Harness■
■   ■■■ mcp-server/                # Python
■   ■■■ decision-service/          # Python
■   ■■■ alert-service/             # Go
■   ■■■ integration-eam/           # Python
■   ■■■ audit-service/             # Go
■■■ packages/
■   ■■■ pdm-core/                  # ■■■■■■■■■■■■■
■   ■■■ pdm-signal/                # ■■■■■■■■■■■
■   ■■■ pdm-testkit/               # ■■■■■■■■■■■
■■■ deploy/
■   ■■■ helm/                      # Chart
■   ■■■ k8s/                       # ■■■■■
■   ■■■ edge/                      # K3s ■■■■■
■   ■■■ argocd/                    # GitOps Application
■■■ tests/
```

```
■   ■■■ unit/  integration/  e2e/  contract/  chaos/  perf/  security/
■■■ datasets/                    # DVC ■■■■■■ Git
■■■ models/                      # ■■■■■■■■■■■■
■■■ .harness/                    # ■■■■■■
```

为什么 Monorepo： 契约（`pdm-core`）变更需要原子性；6 人团队拆多仓的管理成本高于收益。

---

4. 环境与配置管理

4.1 环境矩阵

环境	集群	数据	模型	访问
dev	单节点 K3s	合成	stub	开发
ci	临时	夹具	stub	流水线
test	3 节点	脱敏 1 月	小模型	团队
staging	3 节点	脱敏 3 月	全量	团队 + 业务
prod	生产	真实	全量	受控

4.2 配置分层

```
■■■■■■■■■■
  ■■ ■■■■■■deploy/values-{env}.yaml■
       ■■ ■■■■■■ConfigMap■■■■■■■
            ■■ ■■■■■■■Feature Flag■■■■■■
```

规则： 判据阈值、提示词、技能版本 不属于配置，属于"受控资产"，必须走变更审批（见 docs/07 §5.3）。

4.3 密钥管理

项	方案
存储	Vault / K8s Secret（加密 etcd）
注入	运行时挂载，不写镜像
轮换	90 天自动
审计	每次读取记录
禁止	硬编码、日志打印、异常堆栈泄露

---

5. GitOps（ArgoCD）

```
Git■deploy/■  ←■■ ArgoCD ■■■■■ ■■→  K8s ■■
      ■                                ■
      ■■■ ■■■■■■                       ■■■ ■■■■■■■■■■■
```

规则	说明
禁止 `kubectl apply` 手工改生产	所有变更走 Git
禁止 `latest` tag	用 digest
漂移检测	每 3 分钟，漂移 > 5 分钟告警
同步策略	staging 自动；prod 手动批准
回滚	`argocd app rollback` 或 Git revert

5.1 边缘节点更新策略（关键差异）

边缘不能像中心一样"随时滚"：

```
1. ■■■■ → ■■■■■■■■■■■■■■■■■■
2. ■■■■■■■■■■■■■■■■■
3. ■■■■■■■■■■ 2-4 ■■■■■
4. ■■■■■■■ → ■■ 1 ■ → ■■ 24h → ■■
5. ■■■■■■■■■■■■■■■■■ 2 ■■■■■
```

边缘更新必须"拉"不能"推"——工厂网络经常反向不通。

---

6. 可观测性

6.1 三大支柱

支柱	工具	关键内容
Metrics	Prometheus + Grafana	采集速率、诊断时延、HI 分布、告警量、token 成本
Logs	Loki	结构化 JSON，trace_id 贯穿
Traces	OTel + Tempo/Jaeger	一次诊断的完整调用链

6.2 必埋指标（Golden Signals + AI 特有）

类别	指标
延迟	诊断 P50/P95/P99、工具调用耗时、LLM 首 token 时延
流量	采集点/秒、诊断 QPS、MCP 调用/分钟
错误	工具失败率、LLM 超时率、EAM 失败率
饱和	CPU/内存/磁盘/队列深度
AI 特有	token/次、成本/次、缓存命中率、工具选择错误率、降级触发次数、假完成拦截次数
业务	告警量/天、工单转化率、方案卡采纳率、误报率

6.3 SLO

SLI	SLO	错误预算
诊断 API 可用性	99.5%/月	3.6 小时
诊断 P95 时延	≤ 8 s	5% 请求可超
告警送达时延	≤ 60 s	1% 可超
数据完整率	≥ 99.5%	—

错误预算耗尽 → 冻结功能发布，只修稳定性。 这是 SRE 的硬规则。

---

7. 发布策略

7.1 中心侧：金丝雀

```
5% ■■■1 ■■→ 25%■1 ■■→ 100%
■■■■■■■■■■■■■P95■■■■■■
■■■■■■■ → ■■■■
```

7.2 边缘侧：分批

1 ■■72 ■■■→ 1 ■■■■1 ■■→ ■■

边缘灰度时间必须更长——因为边缘故障的影响是"这台设备没人看着了"，而不是"页面打不开"。

7.3 模型发布（特殊）

步骤	要求
影子模式	新模型并行跑 2 周，只记录不生效，对比差异
小流量	10% 设备
人工复核	新模型的所有橙色以上告警，前 100 条人工复核
全量	无异常后
回滚	保留上一版本，一键切换

---

8. 供应链安全（SLSA Level 3 目标）

要求	实现
构建可复现	固定基础镜像 digest、锁定依赖（lock 文件）
构建隔离	流水线在隔离 runner 执行
来源证明	生成 SLSA provenance
制品签名	cosign（keyless，OIDC）
部署校验	准入控制器验证签名，未签名镜像拒绝
SBOM	Syft 生成，随制品归档
依赖审计	每周生成漏洞报告

---

9. 配置管理（GB/T 8567 要求）

配置项	标识规则	基线
源码	Git commit SHA	每次发布打 tag `v{major}.{minor}.{patch}`
镜像	`registry/pdm/{svc}@{sha256}`	随发布
Helm Chart	语义化版本	随发布
模型	`{name}-{version}-{dataset_hash}`	模型注册表
判据库	`criteria-v{n}`	每次变更
提示词	`prompt-v{n}`	每次变更
技能	`skill-{name}-v{n}`	每次评审通过
文档	与代码同仓	随发布

三库分离原则：代码库（Git）、模型库（Registry）、数据湖（对象存储）各自版本化，通过指纹关联。

---

10. 团队协作规范

实践	规则
分支	主干开发 + 短分支（≤ 2 天）
提交	Conventional Commits（feat/fix/docs/refactor/test/chore）
PR	≤ 400 行；必须 1 人 review；必须过 CI
Code Review	关注：边界条件、错误处理、可观测性、安全
每日	15 分钟站会（只讲阻塞）
每周	技术债梳理 30 分钟
每迭代	回顾 + 1 个改进项落地

10.1 AI 辅助编码的纪律（SDD 配套）

本项目大量使用 AI 编码工具，必须遵守：

- 先规格后代码：没有 `spec.md` 不允许让 AI 生成实现。
- AI 生成代码视同外包代码：必须 review、必须测试、必须能解释。
- 禁止 AI 直接改判据/提示词/技能（这些是受控资产）。
- 提交信息注明 AI 参与度（`assisted-by: copilot`），便于追溯。
- 禁止把生产数据贴进外部 AI 工具。

> 这 5 条不是保守，是 2026 年企业级 AI 编码的真实教训：AI 让写代码变快，让"理解代码"变慢。规格是唯一能对抗这种熵增的东西。